



UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Projeto de Software Web, Móvel e na Nuvem

Bancada

Gestão digital de folhas de obra para oficinas

Primeiro cliente: oficina Duarte & Raposo

Tiago Dias Pereira, n.º 55019

Universidade da Beira Interior

26 de setembro de 2026

Conteúdo

Resumo	4
1 Introdução	5
1.1 Motivação	5
1.2 De uma oficina para qualquer oficina	5
1.3 Objetivos	5
1.4 Como o projeto evoluiu	6
1.5 Organização do relatório	6
2 Engenharia de requisitos	6
2.1 Levantamento a partir das folhas de papel	6
2.2 Quem usa a aplicação	7
2.3 Histórias de utilizador	7
2.4 Requisitos não funcionais	7
3 Modelo de dados	8
3.1 Primeira versão	8
3.2 Segunda versão: uma base de dados para várias oficinas	8
3.3 Normalização e exceções conscientes	10
3.4 Regras garantidas pela própria base de dados	10
4 Arquitetura e infraestrutura	11
4.1 Visão geral	11
4.2 Docker	11
4.3 Configuração e segredos	12
4.4 Organização do repositório	12
4.5 Ferramentas usadas	13
5 A API REST	13
5.1 Organização	13
5.2 Rotas	13
5.3 Regras de negócio	15
5.4 Validação	15
5.5 Transações	15
5.6 Contas no SQL Server	15
6 Segurança	16
6.1 A revisão	16
6.2 Sessões	16
6.3 Isolamento entre oficinas	17
6.4 O modo bancada	17
6.5 Permissões mínimas na base de dados	18
6.6 Outras proteções	18

7	Interface	18
7.1	Ponto de partida	18
7.2	Identidade visual	18
7.3	O ecrã principal: o tablier	19
7.4	O tablet partilhado	19
7.5	Nova entrada e folha de obra	19
7.6	Gestão, computador e telemóvel	21
7.7	Acessibilidade e uso na oficina	21
7.8	Desempenho	23
8	Testes e validação	23
8.1	Testes manuais da primeira fase	23
8.2	Testes automáticos	24
8.3	Integração contínua	24
8.4	Verificação da interface	25
9	Conclusão	25
9.1	O que está feito	25
9.2	O que falta	25
9.3	Reflexão	26
	Referências	27
	Acrónimos	27
	Anexos	29
	Anexo I: Esquema da base de dados (segunda versão)	29
	Anexo II: Diagrama concetual (primeira versão)	30
	Anexo III: Esquema relacional (primeira versão)	31

Lista de Figuras

1	Arquitetura da Bancada.	11
2	Quadro da oficina no tablet (tema claro).	20
3	O quadro no tema escuro, visto por um gestor.	20
4	Modo bancada: “Quem vai trabalhar?” e o teclado do PIN.	21
5	Nova entrada: a matrícula escrita na chapa e o veículo encontrado.	21
6	Folha de obra de uma autocaravana, com as linhas e as contas.	22
7	O quadro no computador do gestor e no telemóvel.	22
8	Esquema relacional da Bancada (segunda versão).	29
9	Diagrama concetual da primeira versão.	30
10	Esquema relacional da primeira versão.	31

Lista de Tabelas

1	As três fases do projeto.	6
2	As seis tabelas da segunda versão.	9
3	Rotas da API.	14
4	Principais problemas de segurança encontrados na revisão e o que mudou.	16
5	Testes manuais da primeira fase.	23
6	Os 52 testes automáticos da API.	24

Lista de Excertos

1	Chaves estrangeiras compostas da tabela <code>Folha_Obra</code>	10
---	---	----

Resumo

A Bancada é uma aplicação web que substitui as folhas de obra em papel das oficinas. Nasceu na Duarte & Raposo, a oficina de mecânica, eletricidade e autocaravanas onde trabalho, e começou por ser um sistema só para ela. A meio do projeto passou a ser uma plataforma para qualquer oficina: cada oficina regista-se sozinha e só vê os seus próprios dados. A Duarte & Raposo ficou como primeiro cliente.

Neste momento o sistema tem um modelo de dados em SQL Server (na segunda versão, já preparada para várias oficinas), uma Application Programming Interface (API) REST em Node.js com Express e uma interface em React pensada para um tablet partilhado na oficina, onde cada mecânico entra com o seu nome e um PIN. O ecrã principal imita as luzes do tablier de um carro: uma luz por estado da reparação, acesa quando há carros nesse estado. A API tem 52 testes automáticos que correm contra uma base de dados SQL Server real, incluindo testes que tentam ler e alterar dados de outra oficina, e o repositório tem integração contínua no GitHub.

Também fica dito o que ainda não foi feito: a aplicação ainda não está instalada na oficina, não foi testada pelos mecânicos e não está publicada na Internet com HTTPS.

Palavras-chave

Folhas de obra; Oficinas automóveis; Aplicação web multi-oficina; React; Node.js; SQL Server; Segurança web; Docker.

1 Introdução

1.1 Motivação

No dia a dia da Duarte & Raposo, uma oficina no Canhoso, na Covilhã, especializada em mecânica, eletricidade e autocaravanas, a entrada dos veículos e o registo das peças são feitos em papel. Cada carro que entra tem uma folha de obra, onde se escreve o que o cliente pediu, as peças que se aplicaram e as horas de trabalho. Funciona, mas tem problemas que se sentem todos os dias: é difícil saber o que se fez a uma autocaravana que cá esteve no ano passado, as folhas sujam-se e perdem-se pela oficina, no fim da reparação perde-se tempo a fazer contas à mão para passar a fatura e nem sempre se sabe quem fez o quê.

Como trabalho lá, conheço estes problemas por dentro. Este projeto foi a forma de juntar o útil ao agradável: usar o que aprendi no curso para resolver um problema real, que me facilita o trabalho a mim e ao resto da equipa. As autocaravanas tiveram atenção especial desde o início, porque são veículos com duas partes (a mecânica do chassis e a célula habitacional) e precisam de um registo mais detalhado do que um ligeiro comum.

1.2 De uma oficina para qualquer oficina

O projeto começou como um sistema só para a Duarte & Raposo. Quando o revi, antes desta entrega, percebi que quase nada nele era específico desta oficina: as folhas em papel, as contas à mão e o histórico perdido são problemas de muitas oficinas pequenas. Decidi por isso transformá-lo numa plataforma para várias oficinas, com a Duarte & Raposo como primeiro cliente, e dei-lhe um nome: Bancada, que é o sítio onde se trabalha numa oficina.

A mudança mexeu com quase tudo. A base de dados passou a ter uma tabela de oficinas e as outras tabelas passaram a indicar a que oficina pertence cada registo. A API passou a filtrar todas as consultas pela oficina de quem tem a sessão. Uma oficina nova pode registar-se sozinha. E apareceu um risco que não existia antes e que passou a ser o mais grave de todos: uma oficina conseguir ver ou alterar os dados de outra. A secção 6 explica como isso foi tratado.

1.3 Objetivos

Os objetivos, tal como estão agora, são estes:

- substituir a folha de obra em papel por uma folha digital, com as peças, a mão de obra e os totais com Imposto sobre o Valor Acrescentado (IVA) calculados sem erros de arredondamento;
- pensar a aplicação para o sítio onde vai ser usada: um tablet partilhado na oficina, por mecânicos com as mãos sujas, com barulho e luz forte;
- dar ao gestor uma visão do que está na oficina e acesso ao histórico de clientes e veículos;
- servir várias oficinas, cada uma só com acesso ao que é seu;
- ter uma base segura e testada, que se põe a correr com poucos comandos (Docker).

Na primeira versão dos objetivos eu falava numa aplicação para o telemóvel dos mecânicos e num *dashboard* para o computador dos gestores. Com o tempo percebi que

o dispositivo certo para os mecânicos é um tablet fixo na oficina, partilhado por todos. A interface foi desenhada a pensar nele, sem deixar de funcionar no computador e no telemóvel.

1.4 Como o projeto evoluiu

O trabalho foi feito em três fases, resumidas na Tabela 1. O repositório tem o histórico completo, commit a commit.

Quando	O que foi feito
junho de 2026	Levantamento de requisitos a partir das folhas de papel, modelo de dados (primeira versão) e base de dados em SQL Server com Docker.
julho de 2026	API para colaboradores, clientes e autocaravanas, com testes manuais (relatório intercalar). Depois: rotas das folhas de obra, login com JWT, primeiras proteções (helmet, limite de tentativas de login, bcrypt) e início da interface em React.
setembro de 2026	Revisão completa de segurança, desempenho e viabilidade. Passagem a multi-oficina (Bancada), segunda versão da base de dados, modo bancada para o tablet partilhado, 52 testes automáticos, integração contínua, interface nova e documentação do projeto.

Tabela 1: As três fases do projeto.

1.5 Organização do relatório

A secção 2 descreve os requisitos e quem usa a aplicação. A secção 3 apresenta as duas versões do modelo de dados. A secção 4 mostra a arquitetura, a infraestrutura e as ferramentas usadas, e a secção 5 a API. A secção 6 trata da segurança e a secção 7 da interface. A secção 8 descreve os testes e a secção 9 fecha com o que está feito, o que falta e o que aprendi.

2 Engenharia de requisitos

2.1 Levantamento a partir das folhas de papel

Como trabalho na oficina, o levantamento foi direto: peguei nas folhas de obra e nas faturas em branco que usamos e tirei delas a informação que tem de passar para o sistema.

- **Cliente:** nome, telefone, Número de Identificação Fiscal (NIF), morada e email. Na primeira versão o NIF e a morada eram obrigatórios. Deixaram de ser, porque muitos clientes de autocaravanas são turistas estrangeiros, com contribuintes de outros países ou sem morada em Portugal. O telefone ficou obrigatório, porque é por ele que se avisa que o carro está pronto.
- **Veículo:** matrícula, tipo, marca, modelo e ano. Nas autocaravanas guarda-se também a marca da célula, separada da marca e do modelo do chassis.

- **Entrada:** data, quilómetros, o que o cliente pediu (observações) e conselhos para uma próxima visita.
- **Serviço:** as linhas de peças, mão de obra e outros custos, cada uma com quantidade, preço unitário e categoria.

2.2 Quem usa a aplicação

- **Mecânicos**, no chão da oficina, num tablet partilhado fixo na bancada ou na parede. Vários mecânicos usam o mesmo tablet ao longo do dia, muitas vezes com as mãos sujas ou de luvas.
- **Gestores** (donos ou responsáveis da oficina), sobretudo no computador, para ver o que se está a fazer, consultar o histórico, gerir a equipa e acompanhar os valores.
- **Oficinas novas**, que se registam sozinhas na plataforma (a oficina e o primeiro gestor).

2.3 Histórias de utilizador

Com base nas rotinas da equipa e nas dificuldades do dia a dia na oficina, escrevi as necessidades como histórias de utilizador:

- *“Como mecânico, quero entrar no tablet da oficina com o meu nome e um PIN, sem escrever emails nem passwords, para começar a trabalhar logo.”*
- *“Como mecânico, quero ver num relance que carros estão na oficina e em que estado.”*
- *“Como mecânico, quero dar entrada a um carro a partir da matrícula: se já cá esteve, aparece logo; se é novo, registo o carro e o dono no mesmo passo.”*
- *“Como mecânico, quero acrescentar as peças e as horas à folha do carro no momento em que as aplico.”*
- *“Como mecânico, preciso de um espaço para conselhos, para avisar o cliente de peças que vão precisar de ser trocadas no futuro.”*
- *“Como gestor, preciso que o sistema some os valores e calcule o IVA sozinho, e que fique registado quem fez cada coisa.”*
- *“Como gestor, quero gerir a equipa (PINs e acessos) e conseguir desligar o tablet se ele se perder.”*
- *“Como responsável de uma oficina nova, quero registar a minha oficina sem ter de falar com ninguém.”*

2.4 Requisitos não funcionais

- **Tablet partilhado e mãos sujas:** alvos de toque grandes (48 píxeis ou mais), poucos passos, pouco para escrever, letra legível a um ou dois metros. Tema claro para a luz forte e tema escuro para o fim do dia.
- **Quem fez fica registado:** a autoria de cada folha e de cada linha vem da sessão, nunca de um campo que se possa escolher.

- **Contas certas ao cêntimo:** totais e IVA calculados no servidor, com números decimais exatos.
- **Isolamento entre oficinas:** uma oficina nunca vê nem altera os dados de outra.
- **Segurança:** passwords guardadas com hash, sessões que não possam ser roubadas por um script da página, proteção contra pedidos forjados e contra tentativas repetidas de login.
- **Histórico:** clientes, veículos e colaboradores são arquivados e nunca apagados.
- **Língua:** português de Portugal.
- **Sem faturação:** em Portugal, as faturas têm de ser emitidas por programas certificados pela Autoridade Tributária e Aduaneira (AT) (Portaria n.º 363/2010). A Bancada calcula os totais e o IVA, e a fatura é emitida no programa de faturação da oficina. A própria interface diz isto ao lado dos totais.
- **Portabilidade:** o sistema tem de correr da mesma forma em qualquer computador (Docker).

3 Modelo de dados

3.1 Primeira versão

Na primeira fase desenhei uma base de dados relacional com cinco tabelas: *Colaboradores*, *Cliente*, *Autocaravana*, *Folha_Obra* e *Linha_reparacao*. Nessa altura o sistema era só para a Duarte & Raposo e os veículos eram só autocaravanas, com a matrícula como chave primária. Já nessa versão tomei três decisões que se mantiveram:

- **Separar o chassis da célula:** as autocaravanas guardam a marca e o modelo do chassis (a parte mecânica) e, à parte, a marca da célula habitacional. Na oficina isto importa, porque as peças de motor e as peças da célula vêm de fornecedores diferentes.
- **Saber quem registou:** cada folha de obra guarda o colaborador que a registou. No papel, as assinaturas perdem-se.
- **Soft delete:** clientes, veículos e colaboradores têm um campo *Ativo*. Nunca se apagam, só se arquivam, para não se perder o histórico nem os dados de garantias.

Os diagramas dessa versão, feitos no ERDPlus, estão nos Anexos II e III. Ao revê-los encontrei dois erros. No esquema relacional, a chave estrangeira entre a folha e as linhas está do lado errado: aparece um *ID_linha* na *Folha_Obra*, o que só permitiria uma linha por folha. No script SQL estava certo (*ID_folha* na *Linha_reparacao*), mas o diagrama não correspondia ao que tinha implementado. E no diagrama concetual a cardinalidade entre colaboradores e folhas ficou invertida: um colaborador regista muitas folhas, e não o contrário. Os dois erros ficaram corrigidos na segunda versão.

3.2 Segunda versão: uma base de dados para várias oficinas

Com a passagem a multi-oficina o esquema foi reescrito. Tem agora seis tabelas, resumidas na Tabela 2. O diagrama completo está no Anexo I e o script, com um comentário a explicar cada decisão, está em `backend/database/schema.sql`.

Tabela	O que guarda
Oficina	Cada oficina da plataforma: nome, NIF (único e validado pelo dígito de controlo), contactos, taxa de IVA, o contador de folhas e a versão da bancada (para desligar os tablets).
Colaborador	Gestores e mecânicos. Entram com email e password, com um PIN no tablet, ou das duas formas. Guarda também as falhas de PIN e a versão da sessão.
Cliente	Os clientes de cada oficina. O telefone é obrigatório; o NIF é opcional e aceita contribuintes estrangeiros.
Veiculo	Qualquer tipo de veículo (ligeiro, comercial, autocaravana, motociclo, pesado, outro). Nas autocaravanas, a marca e o modelo são os do chassis e há a marca da célula.
Folha_Obra	Uma por cada entrada de um veículo: número dentro da oficina, estado, quilómetros, observações, conselhos, taxa de IVA do dia e datas de conclusão e entrega.
Linha_Reparacao	As peças, a mão de obra e os outros custos de cada folha, com quem registou cada linha.

Tabela 2: As seis tabelas da segunda versão.

As decisões mais importantes foram estas:

- **A oficina em todas as tabelas principais.** Colaborador, Cliente, Veiculo e Folha_Obra têm a coluna ID_Oficina. As chaves estrangeiras entre elas são compostas, isto é, incluem sempre a oficina (Excerto 1). Assim é a própria base de dados que impede uma folha da oficina A de apontar para um veículo, um cliente ou um colaborador da oficina B, mesmo que a API tenha um erro. As linhas não têm a coluna: pertencem a uma folha, e a folha já é de uma oficina.
- **Veículos de qualquer tipo.** A tabela Autocaravana passou a Veiculo, com um campo Tipo. A marca da célula continua a existir, e uma restrição CHECK só a deixa preencher nas autocaravanas.
- **Chave artificial nos veículos.** A matrícula deixou de ser a chave primária. As matrículas estrangeiras têm formatos diferentes, e a mesma matrícula pode existir em duas oficinas. A matrícula é guardada normalizada (só letras e números, em maiúsculas), para AA-00-AA, aa 00 aa e AA00AA serem o mesmo carro.
- **Numeração por oficina.** Cada oficina tem a sua sequência de folhas (a folha n.º 1 da Duarte & Raposo e a folha n.º 1 de outra oficina). O número vem de um contador na tabela Oficina, atualizado na mesma transação em que a folha é criada.
- **A folha guarda o que era verdade no dia da entrada.** Guarda o cliente (se o carro mudar de dono, o histórico não muda) e uma cópia da taxa de IVA (se a oficina mudar a taxa, as folhas antigas não mudam).
- **Tipos certos.** NVARCHAR para texto com acentos, DECIMAL para dinheiro e quantidades, datas em DATETIME2 e em UTC.

- **Índices para as consultas reais.** O SQL Server não cria índices nas chaves estrangeiras sozinho, por isso há índices compostos para o quadro da oficina (folhas por estado), para o histórico de um veículo e para as linhas de uma folha.

```

1 CONSTRAINT FK_Folha_Veiculo FOREIGN KEY (ID_Oficina, ID_Veiculo)
2 REFERENCES Veiculo (ID_Oficina, ID_Veiculo),
3 CONSTRAINT FK_Folha_Cliente FOREIGN KEY (ID_Oficina, ID_Cliente)
4 REFERENCES Cliente (ID_Oficina, ID_Cliente),
5 CONSTRAINT FK_Folha_Colaborador FOREIGN KEY (ID_Oficina, ID_Colaborador)
6 REFERENCES Colaborador (ID_Oficina, ID_Colaborador),
7 -- o número da folha é único dentro de cada oficina
8 CONSTRAINT UQ_Folha_Oficina_Numero UNIQUE (ID_Oficina, Numero),

```

Excerto 1: Chaves estrangeiras compostas da tabela Folha_Obra.

3.3 Normalização e exceções conscientes

O modelo está na Terceira Forma Normal (3FN), com três exceções que deixei de propósito:

- o ID_Oficina na Folha_Obra podia ser obtido através do veículo, mas é ele que permite as chaves estrangeiras compostas e que as consultas do quadro filtrem pela oficina sem juntar tabelas;
- o ID_Cliente na Folha_Obra não é redundante, porque guarda o dono no dia da entrada, que pode não ser o dono de hoje;
- a Taxa_IVA na Folha_Obra é o valor histórico, e não uma cópia que tenha de acompanhar a da oficina.

3.4 Regras garantidas pela própria base de dados

Algumas regras não ficam só na API, porque é mais seguro que seja a base de dados a garanti-las:

- restrições CHECK para os valores permitidos (estados da folha, cargos, categorias das linhas, tipos de veículo), para quantidades positivas e preços não negativos, e para regras como “um gestor tem de ter email e password”;
- índices únicos filtrados: a mesma matrícula só existe uma vez entre os veículos ativos de uma oficina, e o mesmo acontece com o NIF dos clientes;
- o login que a API usa na base de dados não tem permissão para apagar oficinas, colaboradores, clientes, veículos nem folhas. O soft delete deixou de depender só do código e passou a ser garantido pelo SQL Server. As linhas podem ser apagadas, porque remover uma linha de uma folha aberta é uma operação normal.

Depois de criada a base de dados, o SQL Server conta 6 tabelas, 21 índices, 14 restrições CHECK e 9 chaves estrangeiras.

4 Arquitetura e infraestrutura

4.1 Visão geral

A Figura 1 mostra as três peças do sistema: o frontend em React, que corre no browser do tablet, do computador ou do telemóvel; a API em Node.js com Express; e a base de dados em SQL Server.

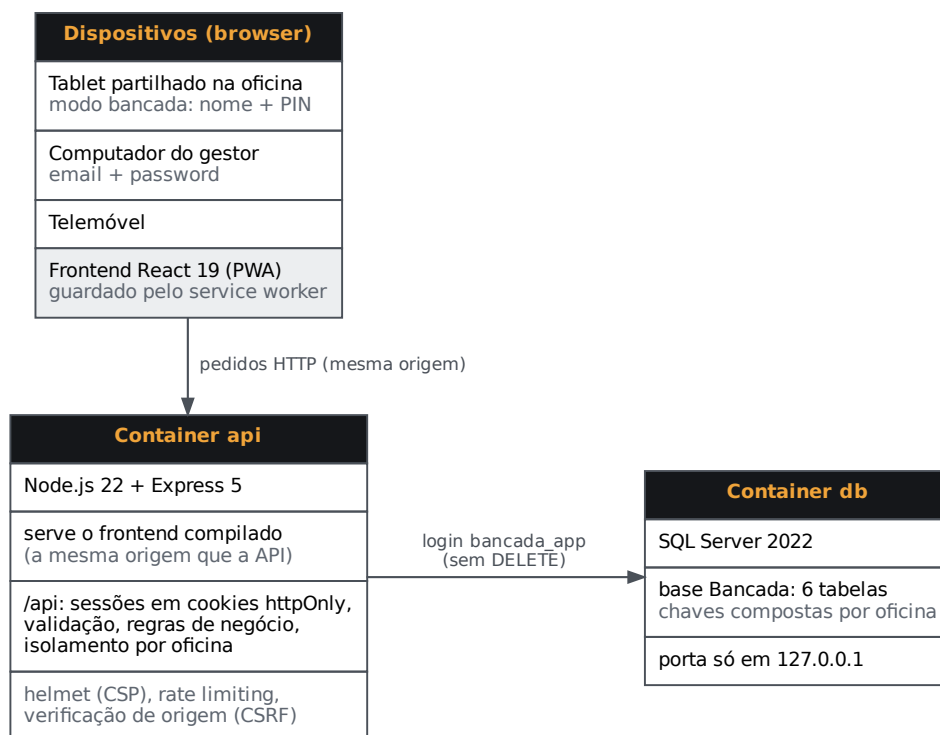


Figura 1: Arquitetura da Bancada, tal como está implementada. Com `docker compose --profile app`, os dois containers correm na mesma máquina. Em desenvolvimento, a API e o frontend correm diretamente com Node.

Em produção, a própria API serve o frontend já compilado, e por isso a interface e a API têm a mesma origem. Isto simplifica muito a segurança: as sessões ficam em cookies com `SameSite=Strict`, não é preciso abrir a API a outros domínios (CORS) e a Content Security Policy (CSP) pode ser rigorosa, só com recursos do próprio site. Em desenvolvimento, o servidor do Vite reencaminha os pedidos `/api` para a API, para o browser ver na mesma uma só origem.

A interface é uma Progressive Web App (PWA): pode ser instalada no ecrã inicial do tablet e o service worker guarda a aplicação no dispositivo, para abrir logo. As respostas da API nunca ficam em cache.

4.2 Docker

Uso Docker desde a primeira fase. O ficheiro `docker-compose.yml` tem dois serviços:

- **db:** o SQL Server 2022, com os dados num volume e uma verificação de saúde (*healthcheck*) para os outros serviços saberem quando está pronto. A porta só é publicada em 127.0.0.1. Com 1433:1433, como estava no início, o SQL Server ficava aberto a toda a rede onde o computador estivesse (a da faculdade, a de um café), e o Docker contorna a firewall do sistema.
- **app:** a API a servir o frontend, construída por um Dockerfile em várias etapas (compila o frontend e instala só as dependências de produção) e a correr com um utilizador sem privilégios. Está num perfil à parte: `docker compose up -d` arranca só a base de dados, para desenvolver, e `docker compose --profile app up -d --build` arranca tudo.

Um pormenor prático: a imagem do SQL Server só existe para processadores x86_64, e em Macs com Apple Silicon corre por emulação. A diretiva `platform: linux/amd64` no `docker-compose.yml` torna isso explícito e evita erros de arquitetura. O primeiro arranque demora um pouco mais, mas funciona da mesma forma no meu computador e num portátil com processador M.

4.3 Configuração e segredos

As passwords e o segredo das sessões ficam num ficheiro `.env`, que nunca vai para o repositório. O repositório tem um `.env.example` com todas as variáveis explicadas, e a API recusa arrancar com um segredo curto ou copiado do exemplo. A password do administrador do SQL Server só é usada pelo Docker e pelo script que prepara a base de dados (`npm run db:setup`). A API usa um login próprio, `bancada_app`, com as permissões mínimas descritas na secção 3.

Aqui ficou uma lição. Na primeira fase, a password do utilizador `sa` e um segredo JSON Web Token (JWT) chegaram a estar no código, num repositório público. Tirei-os depois, mas continuam no histórico do git, e por isso têm de ser tratados como públicos e nunca podem ser reutilizados. Nenhum dos segredos atuais esteve alguma vez no repositório.

4.4 Organização do repositório

```
Projeto_final_UBI/
├── backend/ ..... API: rotas, middleware, validação, scripts da BD, testes
├── frontend/ ..... React + Vite (PWA): páginas, componentes, estilos
├── docs/ ..... análise de segurança, desempenho e viabilidade; imagens
├── Relatorio/ ..... este relatório e os anexos
├── .github/workflows/ ..... integração contínua
├── AI.md ..... mapa do projeto para agentes de IA
├── DESIGN.md ..... sistema visual: cores, tipografia, componentes
├── PRODUCT.md ..... utilizadores e princípios do produto
└── docker-compose.yml, Dockerfile, .env.example
```

Além do README, o repositório tem três documentos pensados para quem vier trabalhar no código a seguir, pessoas ou agentes de IA. O `AI.md` é um mapa do projeto: onde está cada coisa, as regras que não se podem partir (o isolamento entre oficinas, a autoria pela sessão, as contas no SQL, o soft delete) e os comandos. O `PRODUCT.md` descreve os utilizadores e os princípios do produto. O `DESIGN.md` descreve o sistema visual no formato `DESIGN.md` (tokens de cor, tipografia e componentes, com as regras de uso).

Quanto aos ramos: o `main` tem a versão estável, o ramo `dev` serve para integrar o trabalho antes de passar para o `main`, e o trabalho de cada tarefa é feito num ramo próprio. A integração contínua (secção 8) corre em cada *push* para o `main` e o `dev` e em cada *pull request*.

4.5 Ferramentas usadas

Usei o VS Code, o Git e o GitHub, o Docker, o Thunder Client (nos testes manuais da primeira fase), o ERDPlus (para os diagramas da primeira versão) e o Graphviz (para os diagramas deste relatório). Para verificar a interface num browser a sério usei o Playwright.

Na terceira fase usei também o Claude Code, um assistente de programação com inteligência artificial que trabalha diretamente no repositório. Usei-o para rever o projeto (segurança, desempenho e viabilidade) e para escrever boa parte do código novo dessa fase: a passagem a multi-oficina, os testes automáticos e a interface. As decisões de produto foram minhas: o nome, manter e elevar a identidade que já tinha (grafite, âmbar e a luz indicadora), o registo público de oficinas, o tablet partilhado como dispositivo principal e o tablier como ecrã principal. Na interface, o assistente seguiu duas *skills* de design para agentes de IA, a *impeccable* e a *taste-skill*. A *impeccable* inclui uma revisão de design feita por um segundo agente, que avalia capturas dos ecrãs e devolve correções. Houve várias rondas de revisão e correção, até o revisor dar as correções pedidas por resolvidas.

5 A API REST

5.1 Organização

A API está em `backend/` e separa as responsabilidades por pastas:

- `routes/`: um ficheiro por recurso (autenticação, oficinas, colaboradores, clientes, veículos e folhas de obra);
- `middleware/`: autenticação, permissões por cargo, verificação de origem, limites de pedidos e tratamento de erros;
- `lib/`: validação dos dados, sessões, passwords e erros;
- `db.js`: um único *pool* de ligações ao SQL Server, partilhado por toda a aplicação, e uma função para correr várias operações numa transação;
- `scripts/`: preparar a base de dados (`db:setup`) e carregar a Duarte & Raposo com dados de demonstração fictícios (`db:seed`).

A API usa o Express 5, que passa sozinho para o tratamento de erros as exceções das rotas assíncronas. Assim, um erro inesperado responde sempre com uma mensagem em JSON e nunca deixa um pedido pendurado.

5.2 Rotas

A Tabela 3 resume as rotas. Todas estão debaixo de `/api`.

Método	Rota	Para quê
POST	/auth/entrar	Entrar com email e password
POST	/auth/sair	Terminar a sessão
GET	/auth/sessao	Quem está a trabalhar e se o dispositivo é a bancada
POST, DELETE	/auth/bancada	Ativar ou desligar este tablet como bancada (gestor)
GET	/auth/bancada	Nomes de quem pode entrar com PIN neste tablet
POST	/auth/bancada/entrar	Entrar com o PIN
POST	/oficinas	Registo público de uma oficina e do primeiro gestor
GET, PUT	/oficinas/atual	Dados e taxa de IVA da oficina (alterar: gestor)
POST	/oficinas/atual/desligar-tablets	Invalidar todos os tablets da oficina (gestor)
GET, POST	/colaboradores	Equipa (gestor)
PUT, DELETE	/colaboradores/:id	Alterar ou tirar o acesso (gestor)
GET, POST	/clientes	Lista com pesquisa e paginação; criar
GET, PUT, DELETE	/clientes/:id	Ver, alterar, arquivar
GET, POST	/veiculos	Lista com pesquisa por matrícula; criar
GET, PUT, DELETE	/veiculos/:id	Ver (com o histórico), alterar, arquivar
GET	/folhas-obra	Lista com filtros (estado, veículo, pesquisa)
GET	/folhas-obra/resumo	Valores do mês (gestor)
POST	/folhas-obra	Nova entrada (folha, e se preciso o veículo e o cliente)
GET, PATCH	/folhas-obra/:id	Ver a folha com as linhas e os totais; mudar estado e notas
POST	/folhas-obra/:id/linhas	Acrescentar uma linha
DELETE	/folhas-obra/:id/linhas/:linhaId	Remover uma linha

Tabela 3: Rotas da API.

5.3 Regras de negócio

- **Estados da folha:** aberta, em curso, a aguardar peças, pronta e entregue. Quando passa a pronta, guarda-se a data de conclusão; quando passa a entregue, a data de entrega.
- **Uma folha entregue fica fechada:** já não aceita linhas nem alterações. Só um gestor a pode reabrir.
- **Um veículo só tem uma folha aberta de cada vez.** Se já está na oficina, a nova entrada mostra a folha que existe.
- **Autoria:** quem abre a folha e quem regista cada linha vem sempre da sessão. Na versão de julho, o colaborador responsável vinha no corpo do pedido, e qualquer pessoa podia registar obra em nome de outra.
- **Arquivar:** um cliente com veículos ativos ou um veículo que está na oficina não se arquivam.
- **Permissões:** os mecânicos não gerem a equipa, não alteram as definições da oficina e não veem o resumo financeiro.

5.4 Validação

Todos os dados que chegam à API passam por um validador próprio (`lib/validar.js`), que devolve os erros campo a campo, em português, para a interface os mostrar ao lado de cada campo. Alguns exemplos: o NIF das oficinas é validado pelo dígito de controlo; o dos clientes aceita contribuintes estrangeiros com o código do país; as matrículas são normalizadas; os valores aceitam a vírgula portuguesa (12,50) e recusam mais de duas casas decimais; os textos são limpos de espaços a mais e de caracteres de controlo e têm um tamanho máximo. Antes desta fase, um ano com letras dava um erro 500, e o cargo era texto livre (dava para inventar cargos).

5.5 Transações

Uma nova entrada pode criar, de uma só vez, o cliente, o veículo, a folha e as primeiras linhas. Tudo isso corre numa transação: ou fica tudo gravado, ou nada. O número da folha também é tirado dentro da transação, e por isso uma entrada que falha não gasta número (há um teste para isto). Acrescentar uma linha é uma única instrução `INSERT ... SELECT` que só insere se a folha for da oficina da sessão e não estiver entregue, sem janela entre a verificação e a escrita.

5.6 Contas no SQL Server

Os totais são calculados no SQL Server com `DECIMAL`, que é exato. Em JavaScript, `3 * 19.99` dá `59.970000000000006`, e um valor assim não pode aparecer numa conta de um cliente. Cada linha é arredondada ao cêntimo e o IVA é calculado sobre o subtotal, com a taxa guardada na folha. A resposta traz os totais por categoria (peças, mão de obra e outros), o subtotal, o IVA e o total. Acrescentar ou remover uma linha devolve logo os totais novos, sem outro pedido.

6 Segurança

6.1 A revisão

No início da terceira fase foi feita uma revisão de segurança de todo o repositório. O *backend* estava bem encaminhado (consultas parametrizadas, passwords com `bcrypt`, `helmet`, limite de tentativas no login e segredos fora do código), mas havia falhas importantes. A Tabela 4 resume as principais. A análise completa, com todos os pontos, está em `docs/analise.md`.

O que encontrei	Risco	O que mudou
Uma oficina ver ou alterar os dados de outra (risco novo, com a passagem a multi-oficina).	Crítico	Filtro por oficina em todas as consultas, chaves estrangeiras compostas e testes automáticos (secção 6.3).
A password <code>sa</code> e um segredo JWT estiveram no código, num repositório público.	Crítico	Fora do código; tratados como públicos e nunca reutilizados.
O SQL Server estava publicado em todas as interfaces de rede.	Alto	Porta só em <code>127.0.0.1</code> .
A API ligava-se à base de dados como administrador (<code>sa</code>).	Alto	Login <code>bancada_app</code> , só com leitura e escrita de dados e sem permissão para apagar.
O responsável de uma folha vinha no pedido e podia ser falsificado.	Alto	A autoria vem sempre da sessão.
O token de sessão ficava no <code>localStorage</code> , ao alcance de qualquer script (XSS).	Médio	Cookie <code>httpOnly</code> e <code>SameSite=Strict</code> , e CSP rigorosa.
Com cookies, passa a haver risco de pedidos forjados (CSRF).	Médio	Verificação da origem em todos os pedidos que alteram dados.
Uma sessão continuava válida horas depois de a conta ser desativada.	Médio	A API relê o colaborador em cada pedido; desativar ou mudar a password corta as sessões logo.
O limite de tentativas de login era só por IP, e numa oficina todos os dispositivos saem com o mesmo IP.	Médio	Limite por IP e por conta, mais um teto por IP.
Validação mínima dos dados.	Médio	Validador para todos os campos e <code>CHECK</code> na base de dados.

Tabela 4: Principais problemas de segurança encontrados na revisão e o que mudou.

6.2 Sessões

Quando alguém entra, a API cria um JWT assinado com HS256 e guarda-o num cookie com duas marcas. Com `httpOnly`, o JavaScript da página não o consegue ler, nem mesmo um script injetado num ataque de Cross-Site Scripting (XSS). Com `SameSite=Strict`, o browser não o envia em pedidos que venham de outros sites. O token tem emissor

e audiência fixos, e o algoritmo também é fixo, para um token de outro tipo ou sem assinatura (`alg: none`) não ser aceite. Há testes que tentam estes ataques.

Em cada pedido, a API volta a ler o colaborador na base de dados e compara a versão da sessão guardada no token com a que está na tabela. Mudar a password, mudar o PIN ou desativar uma conta aumenta essa versão, e todas as sessões antigas dessa pessoa deixam de valer na hora. Quem mudou a própria password recebe logo uma sessão nova. Uma sessão dura 12 horas, o tempo de um turno longo.

Contra os ataques de Cross-Site Request Forgery (CSRF), em que outro site tenta fazer pedidos em nome de quem tem a sessão aberta, há uma segunda proteção por cima do `SameSite=Strict`. Todos os pedidos que alteram dados passam por uma verificação de origem: os cabeçalhos `Origin` e `Sec-Fetch-Site` têm de indicar que o pedido veio da própria aplicação. Um pedido vindo de outro site recebe 403.

6.3 Isolamento entre oficinas

O isolamento está em três camadas, para um erro numa delas não chegar para expor dados:

1. **Na API:** a oficina vem sempre da sessão, e todas as consultas filtram por ela. Pedir um cliente de outra oficina dá 404, tal como pedir um cliente que não existe (não se fica a saber que existe).
2. **Na base de dados:** as chaves estrangeiras compostas impedem uma folha de apontar para dados de outra oficina.
3. **Nos testes:** criam duas oficinas, A e B, e verificam que B não lê, não altera, não arquiva e não usa nos seus registos os dados de A, e que não vê nem altera a equipa de A.

6.4 O modo bancada

O tablet partilhado precisava de uma forma rápida de trocar de pessoa. Escrever email e password com as mãos sujas, várias vezes por dia, não ia funcionar. A solução foi o modo bancada:

- um gestor, já com sessão iniciada, ativa o tablet como bancada da oficina. O tablet recebe um cookie de dispositivo, válido cerca de seis meses, e a sessão do gestor termina;
- a partir daí o tablet mostra “Quem vai trabalhar?” com os nomes da equipa, e cada mecânico entra com o seu nome e um PIN de 4 a 6 algarismos;
- o que o mecânico regista fica em nome dele. Quando acaba toca em Terminar, e se o tablet ficar 5 minutos parado a sessão termina sozinha.

Um PIN é fraco por natureza, e por isso o modo bancada tem limites apertados. O PIN só funciona num tablet ativado por um gestor dessa oficina. Ao fim de 5 falhas o PIN fica bloqueado durante 5 minutos, com a contagem na base de dados e não na memória do servidor. Os PINs óbvios (1234, 0000) são recusados. Uma sessão aberta com PIN não dá acesso à gestão de contas nem às definições da oficina, mesmo que seja de um gestor. E se um tablet se perder, o gestor carrega em “Desligar todos os tablets” e todos os dispositivos da oficina deixam de servir como bancada.

6.5 Permissões mínimas na base de dados

O script `db:setup` cria o login `bancada_app` só com os papéis de leitura e escrita de dados, e nega-lhe explicitamente o `DELETE` nas tabelas com `soft delete`. Se um erro na API, ou um ataque, tentar apagar clientes ou folhas, é o SQL Server que recusa, e há um teste que confirma isto.

6.6 Outras proteções

- **Passwords:** `bcrypt` com custo 12, mínimo de 10 caracteres, máximo de 72 bytes (o `bcrypt` ignora em silêncio o que passa disso) e uma lista de passwords óbvias recusadas.
- **Login com tempo constante:** a API compara sempre a password com um hash, mesmo quando o email não existe. Assim o tempo de resposta não denuncia que contas existem, e a mensagem de erro é a mesma nos dois casos.
- **Limites de pedidos:** no login (por IP e por conta), no registo de oficinas, no PIN e na API em geral.
- **Cabeçalhos:** o `helmet` define a CSP, que só aceita scripts, estilos, fontes e imagens do próprio site, e outros cabeçalhos de segurança. As respostas da API levam `Cache-Control: no-store`.
- **Respostas:** as colunas devolvidas são sempre explícitas. Nenhuma resposta traz passwords, PINs ou hashes, e há um teste para isso.
- **Dependências:** os *lockfiles* estão no repositório, e o `npm audit` corre na integração contínua. Neste momento não há vulnerabilidades conhecidas.

7 Interface

7.1 Ponto de partida

Em julho comecei a interface em React: o ecrã de entrada, com uma luz indicadora que pisca enquanto espera pela resposta do servidor e fica verde ou vermelha consoante o resultado, e a ligação à API. Mas o frontend não arrancava (faltava o ficheiro de entrada, `main.jsx`, e havia ficheiros em pastas trocadas) e os ecrãs da oficina e de gestão ainda não existiam. Na terceira fase a interface foi feita de novo, mantendo a ideia da luz indicadora.

A interface usa React 19, Vite e React Router, com CSS próprio organizado em *tokens* (cores, tipos, espaços). Não usa bibliotecas de componentes.

7.2 Identidade visual

Mantive e elevei a identidade que já tinha: o grafite, o âmbar e a luz indicadora. A ideia que organiza tudo é o tablier de um carro, que tem uma parte sempre escura (o painel de instrumentos, com as luzes de aviso) e uma superfície de trabalho que segue o tema do dispositivo, clara ao sol e escura à noite. Os elementos principais são estes:

- **As luzes de aviso.** Cada estado tem um pictograma desenhado de propósito, cheio, como as luzes de um carro: a frente de um carro (aberta), uma chave de bocas (em curso), um pistão (a aguardar peças), uma bandeira de xadrez (pronta) e uma chave

(entregue). Numa primeira versão, “a aguardar peças” era uma roda dentada, mas a revisão de design notou que toda a gente a lê como “definições”, e foi trocada pelo pistão.

- **A matrícula como chapa.** As matrículas aparecem desenhadas como a chapa: brancas nos dois temas, com letras pretas e a faixa azul europeia. Na chapa grande, as matrículas portuguesas levam as estrelas e o P. É assim que numa oficina se reconhece um carro, mais depressa do que pelo nome do dono.
- **Um só acento.** O âmbar é a única cor de ação: o botão principal de cada ecrã, o separador ativo e o filtro ativo.
- **Letra de sinalética.** A Barlow, que nasceu da sinalética e das matrículas, na versão condensada para títulos, números e chapas. As fontes vêm da própria aplicação e não de um serviço externo.

Todas estas regras estão escritas no `DESIGN.md`, com os valores exatos, para quem continuar o trabalho não ter de os adivinhar.

7.3 O ecrã principal: o tablier

Considereei várias formas para o ecrã principal: um quadro kanban com uma coluna por estado, um quadro de chaves como o que há em muitas oficinas, começar sempre pela matrícula, ou a lista e a folha lado a lado. Escolhi o tablier (Figura 2): uma faixa escura com quatro luzes (abertas, em curso, a aguardar peças e prontas), cada uma acesa quando há carros nesse estado e com o número por baixo. Por baixo fica a pesquisa, o botão “Nova entrada” e a lista dos carros na oficina, dos que estão há mais tempo para os mais recentes. Cada linha mostra a chapa, o veículo e o cliente, o que o cliente pediu e o estado.

Tocar numa luz filtra a lista, e tocar outra vez tira o filtro. Quando alguém entra, as quatro luzes acendem por ordem e ficam acesas só as que têm carros, como num carro quando se roda a chave. Quando um estado passa de zero para um ou mais carros, a luz acende com um pequeno cintilar. O quadro volta a pedir os dados de 20 em 20 segundos, para o tablet mostrar o que os colegas mudaram noutros dispositivos. A Figura 3 mostra o quadro no tema escuro, desta vez visto por um gestor.

7.4 O tablet partilhado

A Figura 4 mostra os dois ecrãs do modo bancada: a escolha de quem vai trabalhar e o teclado do PIN, com teclas de 72 píxeis para dedos com luvas. O teclado também funciona com um teclado físico.

7.5 Nova entrada e folha de obra

A nova entrada começa pela matrícula, escrita num campo com a forma da própria chapa (Figura 5). Enquanto se escreve, aparecem os veículos que já cá estiveram. Se o carro já está na oficina, a aplicação diz em que folha. Se é novo, regista-se o veículo e o dono no mesmo passo.

A folha de obra (Figura 6) junta o que estava no papel. Tem a chapa, o veículo, o cliente com o telefone (tocar liga), a entrada e quem a abriu. Tem o estado, escolhido com um toque, com o mesmo pictograma das luzes aceso no estado atual. E tem as linhas com os totais, as notas e os conselhos. Depois de acrescentar uma linha, o cursor volta à

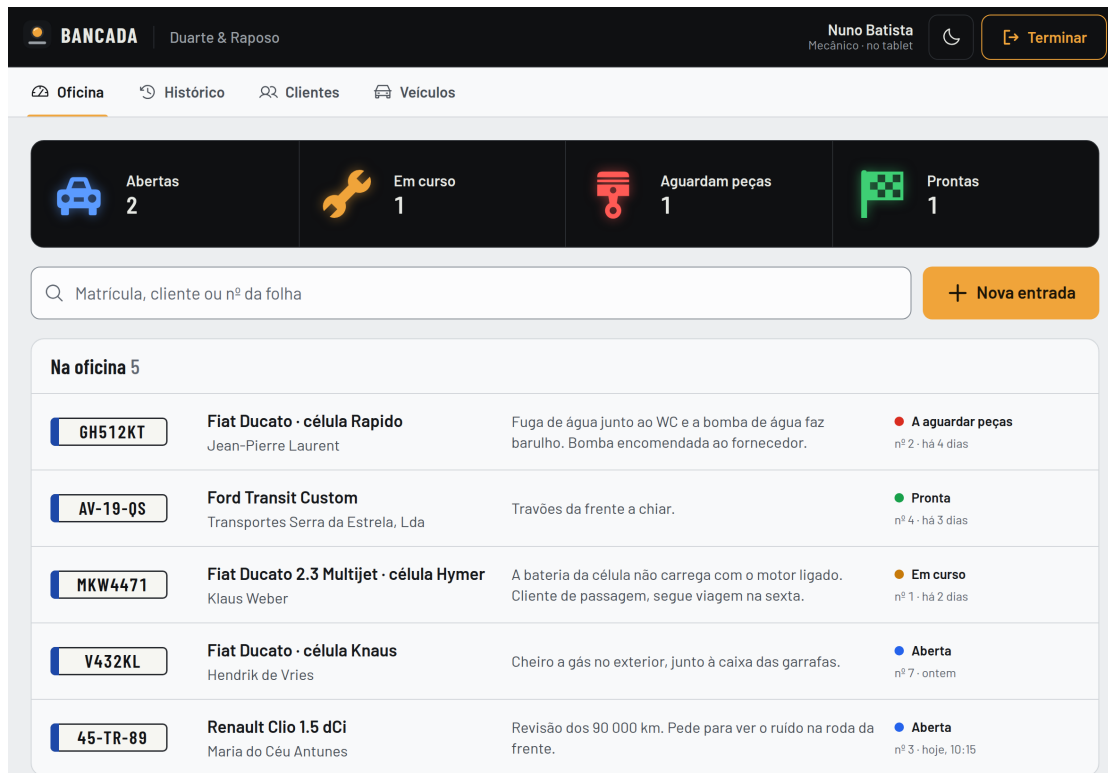


Figura 2: Quadro da oficina no tablet, com um mecânico na bancada (tema claro). Os dados são fictícios.

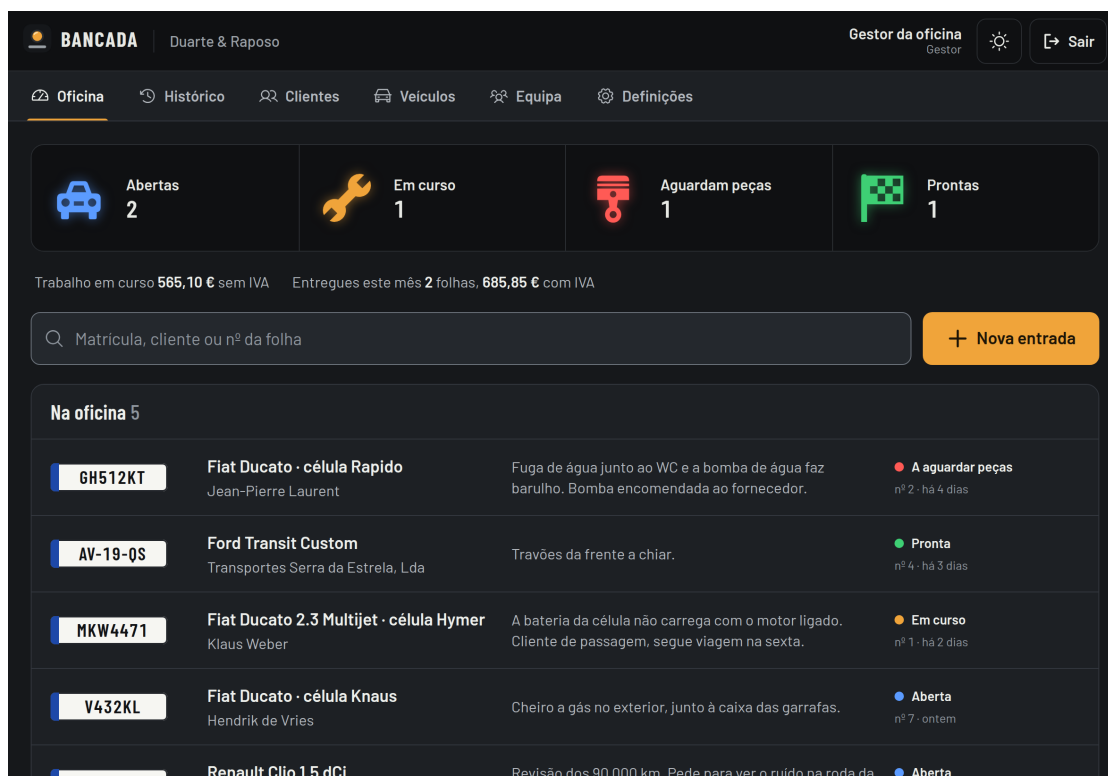


Figura 3: O quadro no tema escuro, visto por um gestor: tem também os valores do mês e os separadores Equipa e Definições. O painel das luzes é escuro nos dois temas.

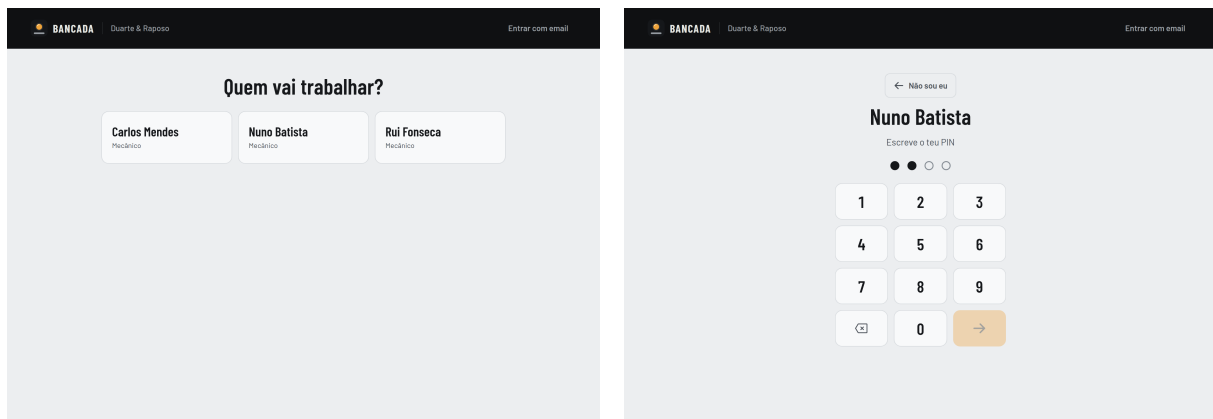


Figura 4: Modo bancada: “Quem vai trabalhar?” e o teclado do PIN.

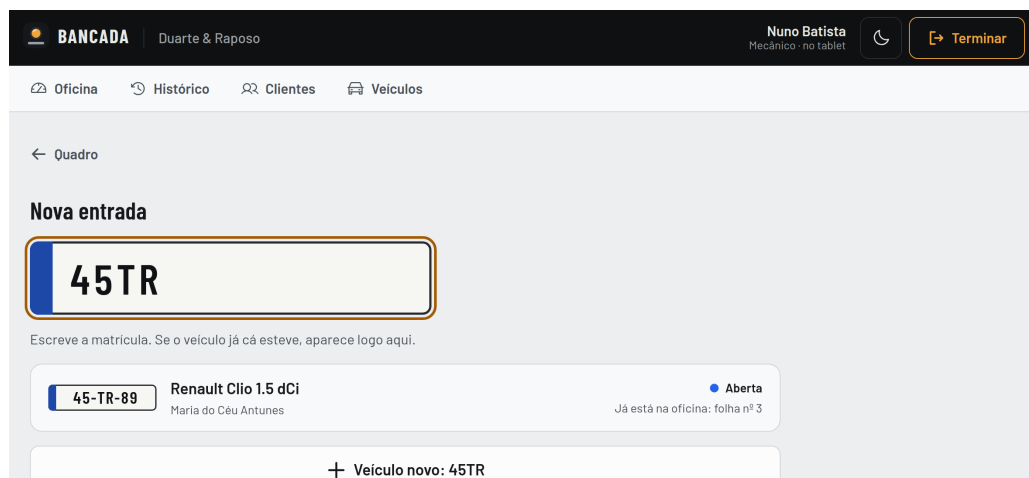


Figura 5: Nova entrada: a matrícula escrita na chapa e o veículo encontrado.

descrição, para quem regista várias peças seguidas não ter de tocar em nada. Remover uma linha pede confirmação no próprio botão, sem janelas por cima.

7.6 Gestão, computador e telemóvel

No computador, o gestor tem os mesmos ecrãs e mais três coisas: os valores do mês por cima do quadro (trabalho em curso sem IVA e folhas entregues com IVA), a equipa (criar colaboradores, definir PINs e tirar acessos) e as definições da oficina (dados, taxa de IVA e tablets). Há também o histórico de folhas, os clientes e os veículos, com pesquisa e o histórico de visitas de cada veículo. A Figura 7 mostra o quadro no computador e no telemóvel.

7.7 Acessibilidade e uso na oficina

- **Contraste:** um script confirma que todas as combinações de texto e fundo passam o nível AA das Web Content Accessibility Guidelines (WCAG) 2.2 (4,5:1) nos dois temas.
- **Toque:** todos os botões e ligações têm pelo menos 48 píxeis, as ações principais do tablet 56 e as teclas do PIN 72. Os efeitos de passar o rato só existem em dispositivos com rato, para não ficarem “presos” depois de um toque.

BANCADA

Duarte & Raposo

Nuno Batista

Mecânico - no tablet

Terminar

Oficina

Histórico

Clientes

Veículos

← Quadro

MKW4471

Folha nº 1

Fiat Ducato 2.3 Multijet · célula Hymer · 2019

Cliente

Klaus Weber

+49 171 5550199

Entrada

há 2 dias, 84 210 km

Aberta por

Nuno Batista

Aberta

Em curso

A aguardar peças

Pronta

Entregue

Peças e serviços

Descrição	Otd.	Preço	Total	
Bateria AGM 100 Ah (célula) Peça · Nuno Batista	1 un.	189,90 €	189,90 €	
Relé separador de baterias 12 V Peça · Nuno Batista	1 un.	64,50 €	64,50 €	
Diagnóstico e substituição Mão de obra · Nuno Batista	2 h	38,00 €	76,00 €	

Adicionar

Contas

Peças	254,40 €
Mão de obra	76,00 €
Outros	0,00 €
Subtotal	330,40 €
IVA (23%)	75,99 €
Total	406,39 €

A fatura é emitida no programa de faturação certificado da oficina.

Figura 6: Folha de obra de uma autocaravana, com as linhas e as contas.

BANCADA

Duarte & Raposo

Gestor da oficina

Sair

Oficina

Histórico

Clientes

Veículos

Equipa

Definições

Abertas

2

Em curso

1

Aguardam peças

1

Prontas

1

Trabalho em curso 565,10 € sem IVA

Entregues este mês 2 folhas, 685,85 € com IVA

Matrícula, cliente ou nº da folha

Nova entrada

Na oficina 5

6H512KT	Fiat Ducato · célula Rapido Jean-Pierre Laurent	Fuga de água junto ao WC e a bomba de água faz barulho. Bomba encomendada ao fornecedor.	A aguardar peças nº 2 - há 4 dias
AV-19-QS	Ford Transit Custom Transportes Serra da Estrela, Lda	Travões da frente a chiar.	Pronta nº 4 - há 3 dias
MKW4471	Fiat Ducato 2.3 Multijet · célula Hymer Klaus Weber	A bateria da célula não carrega com o motor ligado. Cliente de passagem, segue viagem na sexta.	Em curso nº 1 - há 2 dias
V432KL	Fiat Ducato · célula Knaus Hendrik de Vries	Cheiro a gás no exterior, junto à caixa das garrafas.	Aberta nº 7 - ontem
45-TR-89	Renault Clio 1.5 dCi Mário do Céu Antunes	Revisão dos 90 000 km. Pede para ver o ruído na roda da frente.	Aberta nº 3 - hoje, 10:15

BANCADA

Gestor da...

Sair

Oficina

Histórico

Clientes

Abertas

2

Em curso

1

Aguardam peças

1

Prontas

1

Trabalho em curso 565,10 € sem IVA

Entregues este mês 2 folhas, 685,85 € com IVA

Matrícula, cliente ou nº da folha

Nova entrada

Na oficina 5

6H512KT	Fiat Ducato · célula Rapido Jean-Pierre Laurent	Fuga de água junto ao WC e a bomba de água...	A aguardar peças nº 2 - há 4 dias
AV-19-QS			Pronta nº 4 - há 3 dias

Figura 7: O quadro no computador do gestor e no telemóvel.

- **A cor nunca é a única pista:** cada estado tem sempre o nome escrito ao lado da luz.
- **Teclado e leitores de ecrã:** o foco é sempre visível, há uma ligação para saltar para o conteúdo e os campos têm etiquetas (o texto de exemplo nunca substitui a etiqueta).
- **Movimento:** as animações são curtas e desaparecem para quem pede menos movimento no sistema.

7.8 Desempenho

- O JavaScript inicial tem 120 KB comprimido (React DOM 66 KB, React Router 31 KB, a aplicação e os ícones 23 KB) e o CSS 8,4 KB. As páginas de gestão são carregadas à parte, e o browser vai buscá-las com antecedência quando está parado.
- Os ícones da biblioteca Phosphor traziam seis pesos cada um, cerca de 30 KB, mais do dobro do código da aplicação. Um script gera um módulo só com os ícones e os pesos usados, o que tirou 22 KB ao pacote inicial.
- As fontes estão em woff2, só com os caracteres latinos. Os ficheiros com *hash* ficam em cache “para sempre”, e a API comprime as respostas.
- Ao voltar ao quadro, a lista aparece logo, a partir da última resposta guardada em memória, e é atualizada por trás. Essa memória é limpa sempre que a sessão muda, para quem entra a seguir no tablet nunca ver o que ficou da pessoa anterior.

8 Testes e validação

8.1 Testes manuais da primeira fase

Na primeira fase testei a API à mão, com o Thunder Client e com `curl`, contra o SQL Server a correr em Docker. A Tabela 5 mostra os cenários dessa altura. As rotas das autocaravanas deram lugar às dos veículos, e estes cenários estão agora cobertos pelos testes automáticos.

Cenário	Resultado esperado (e obtido)
Criar um cliente com todos os campos válidos	201, com o registo criado
Listar clientes	200, só os clientes ativos
Criar uma autocaravana com um cliente inexistente	400, com uma mensagem clara (sem o erro do SQL)
Criar uma autocaravana com uma matrícula que já existe	409
Criar um colaborador	201, sem a password na resposta
Desativar um colaborador e listar	Não aparece na lista, mas continua na base de dados

Tabela 5: Testes manuais da primeira fase.

8.2 Testes automáticos

Na terceira fase foram escritos 52 testes automáticos com o `node:test`, o executor de testes que já vem com o Node.js. Os testes arrancam a API de verdade e fazem pedidos HTTP com cookies, como um browser, contra um SQL Server real, numa base de dados própria (`Bancada_Teste`) criada de novo em cada ficheiro. Nunca tocam na base de dados de desenvolvimento. A Tabela 6 resume o que cobrem.

Ficheiro	Testes	O que verifica
<code>auth.test.js</code>	16	Registo de oficinas (NIF inválido ou repetido, email repetido, passwords fracas); login com a mesma mensagem para email inexistente e password errada; tokens adulterados e <code>alg: none</code> ; CSRF; sessões cortadas ao desativar ou mudar a password; permissões dos mecânicos; limites de tentativas; respostas sem passwords, PINs nem hashes; cabeçalhos de segurança.
<code>isolamento.test.js</code>	7	A oficina B não lê, não altera, não arquiva e não usa os dados da oficina A, nem vê a equipa de A; a mesma matrícula e o mesmo NIF podem existir em oficinas diferentes; numeração por oficina.
<code>folhas.test.js</code>	11	Totais e IVA ao cêntimo; autoria pela sessão; uma folha aberta por veículo; estados e datas; folha entregue fechada; mudar a taxa de IVA não altera folhas abertas; criar com linhas é tudo ou nada; uma entrada recusada não gasta número; filtros do quadro; regras de arquivo.
<code>bancada.test.js</code>	9	Mecânicos só com PIN; ativar o tablet; o trabalho fica em nome de quem entrou; bloqueio ao fim de 5 falhas; o PIN não serve sem tablet ativado nem noutra oficina; com PIN não se mexe em contas; desligar todos os tablets; a base de dados recusa apagar registos protegidos.
<code>validar.test.js</code>	9	NIF (dígito de controlo e estrangeiros), matrículas, decimais com vírgula, textos, passwords, PINs, pesquisas com caracteres especiais e erros com o caminho do campo.

Tabela 6: Os 52 testes automáticos da API.

Os 52 testes passam e correm em cerca de 22 segundos. Também apanharam erros durante o desenvolvimento: por exemplo, o número 123456789 estava a ser usado como exemplo de NIF inválido, mas tem o dígito de controlo certo.

8.3 Integração contínua

O GitHub Actions corre dois trabalhos em cada *push* para o `main` e o `dev` e em cada *pull request*. O primeiro arranca um SQL Server num container, com passwords geradas na hora que nunca ficam escritas no repositório, verifica as dependências da API com o `npm audit` e corre os testes. O segundo verifica as dependências do frontend, corre o *linter* (oxlint) e compila a aplicação.

8.4 Verificação da interface

A interface foi verificada num browser a sério (Chromium, com o Playwright), com a versão compilada servida pela própria API e com a CSP ligada. O percurso passa por todos os ecrãs no computador (1440 píxeis), no tablet deitado (1180 × 820) e ao alto (820 × 1180) e no telemóvel (390 píxeis), nos temas claro e escuro, incluindo o modo bancada com PIN. No último percurso não houve erros na consola nem violações da CSP. A revisão de design (secção 4) usou as capturas deste percurso.

Falta a validação mais importante: pôr os mecânicos da Duarte & Raposo a usar a aplicação no dia a dia. Isso ainda não foi feito.

9 Conclusão

9.1 O que está feito

O projeto começou com um problema concreto, as folhas de obra em papel da Duarte & Raposo, e chegou a uma plataforma para várias oficinas. Neste momento existe:

- uma base de dados em SQL Server, na segunda versão, com o isolamento entre oficinas garantido também pelas chaves estrangeiras e com as regras principais em restrições da própria base de dados;
- uma API REST com sessões seguras, validação de todos os dados, contas exatas, transações, o modo bancada para o tablet partilhado e o registo público de oficinas;
- uma interface em React, instalável como PWA, pensada para o tablet da oficina e que funciona também no computador e no telemóvel, com temas claro e escuro;
- 52 testes automáticos contra um SQL Server real e integração contínua no GitHub;
- a documentação: README, análise de segurança, desempenho e viabilidade, o mapa para agentes de IA (AI.md) e o sistema visual (DESIGN.md).

9.2 O que falta

Há coisas que ainda não estão feitas, e é justo dizê-las:

- a aplicação ainda não está instalada na oficina nem foi testada pelos mecânicos. É o passo mais importante que falta;
- não está publicada na Internet. Para isso é preciso HTTPS (com cookies marcadas como **Secure**), ligação encriptada à base de dados e cópias de segurança automáticas;
- o esquema da base de dados recria-se do zero. Antes de haver dados reais, é preciso um sistema de migrações;
- não há recuperação de password por email, nem um registo completo de quem alterou o quê (só a autoria das folhas e das linhas);
- a Bancada não emite faturas. O caminho realista é integrar com programas de faturação certificados que têm API, em vez de certificar a própria aplicação;
- para ser usada por outras oficinas a sério, precisa de política de privacidade e de contratos de tratamento de dados com cada oficina, por causa do Regulamento Geral sobre a Proteção de Dados (RGPD).

9.3 Reflexão

A parte mais difícil deste projeto não foi escrever código, foi decidir o que o sistema devia ser. Começou como “uma aplicação para a minha oficina” e acabou como “uma plataforma para oficinas”, e essa mudança obrigou-me a pensar em coisas que no início nem me passavam pela cabeça, como o isolamento dos dados entre oficinas. Também aprendi a desenhar para o sítio onde a aplicação vai ser usada: um tablet partilhado, numa oficina com barulho, luz forte e mãos sujas, é muito diferente de um computador num escritório.

Aprendi ainda que a segurança não é um passo que se faz no fim. A password que deixei no código na primeira fase continua no histórico do repositório, e a única solução é nunca mais a usar. Os testes automáticos dão confiança para mexer no código sem medo de partir o que já funcionava, e durante esta fase apanharam erros que de outra forma teriam passado.

Falta a prova mais importante, que é pôr a Bancada nas mãos dos mecânicos da Duarte & Raposo e ver se é mesmo mais rápida do que o papel. Um dos princípios que segui foi precisamente esse: se um passo digital for mais lento do que escrever na folha, está errado.

Referências

- Docker. *Docker Compose overview*. <https://docs.docker.com/compose/>
- Microsoft. *SQL Server on Linux*. <https://learn.microsoft.com/en-us/sql/linux/>
- Express. *Express 5 documentation*. <https://expressjs.com/>
- React. *React documentation*. <https://react.dev/>
- MDN Web Docs. *Progressive web apps*. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
- MDN Web Docs. *Content Security Policy (CSP)*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>
- MDN Web Docs. *Set-Cookie: SameSite*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie>
- OWASP. *Cross-Site Request Forgery Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- OWASP. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. <https://www.w3.org/TR/WCAG22/>
- Portaria n.º 363/2010, de 23 de junho: certificação prévia dos programas informáticos de faturação.
- Regulamento (UE) 2016/679 (Regulamento Geral sobre a Proteção de Dados).
- Google Labs. *DESIGN.md*. <https://github.com/google-labs-code/design.md>
- VoltAgent. *awesome-design-md*. <https://github.com/VoltAgent/awesome-design-md>
- Paul Bakaus. *impeccable*. <https://github.com/pbakaus/impeccable>
- Leonxlnx. *taste-skill*. <https://github.com/Leonxlnx/taste-skill>

Acrónimos

3FN	Terceira Forma Normal
API	Application Programming Interface
AT	Autoridade Tributária e Aduaneira
CSP	Content Security Policy
CSRF	Cross-Site Request Forgery
IVA	Imposto sobre o Valor Acrescentado
JWT	JSON Web Token
NIF	Número de Identificação Fiscal

PWA Progressive Web App

RGPD Regulamento Geral sobre a Proteção de Dados

WCAG Web Content Accessibility Guidelines

XSS Cross-Site Scripting

Anexos

Anexo I: Esquema da base de dados (segunda versão)

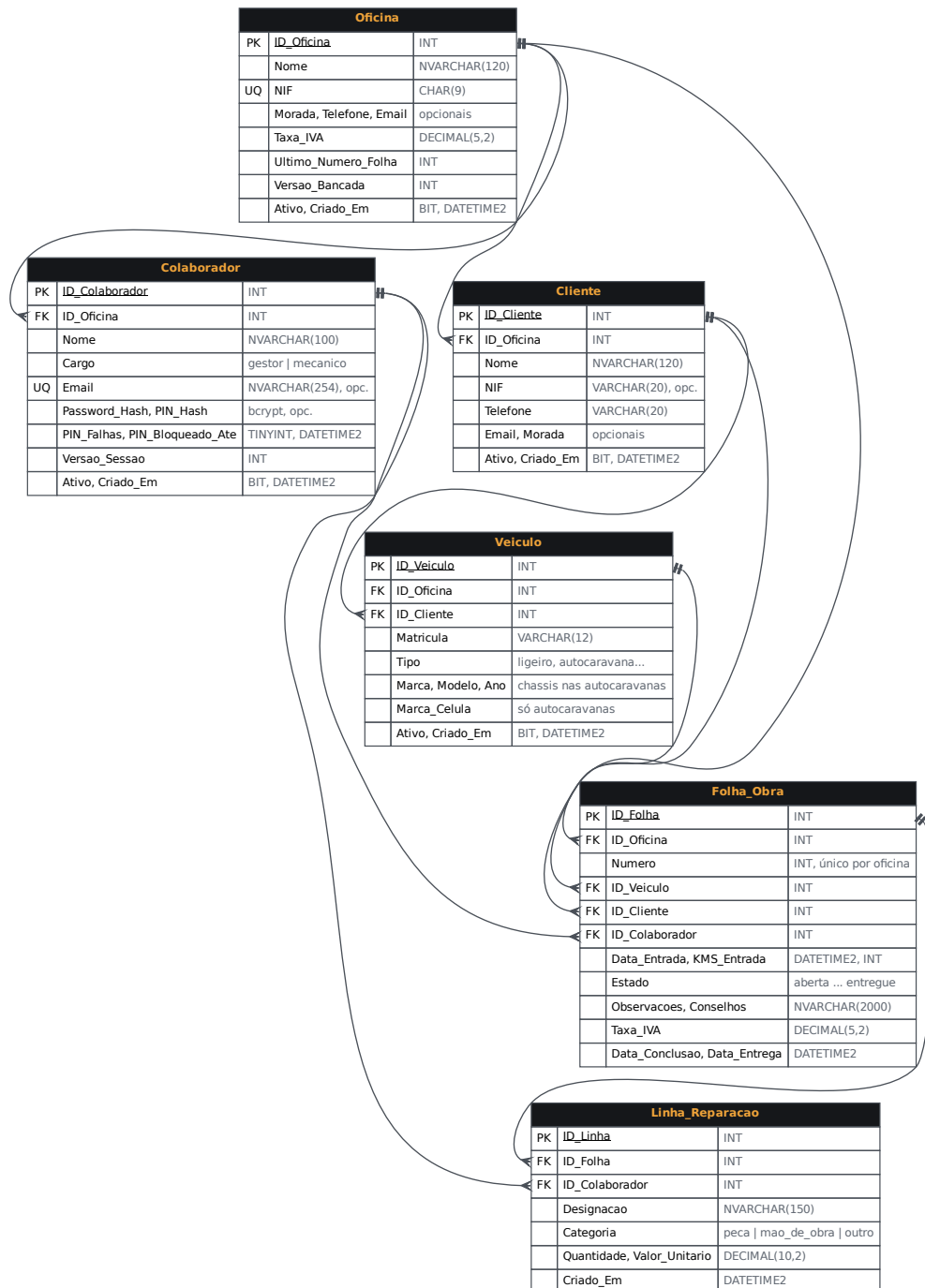


Figura 8: Esquema relacional da Bancada (segunda versão, multi-oficina), gerado com o Graphviz a partir do `schema.sql`. PK: chave primária; FK: chave estrangeira; UQ: única. Cada linha liga uma chave estrangeira à tabela referenciada: um traço do lado “um” e um pé de galinha do lado “muitos”.

Anexo II: Diagrama concetual (primeira versão)

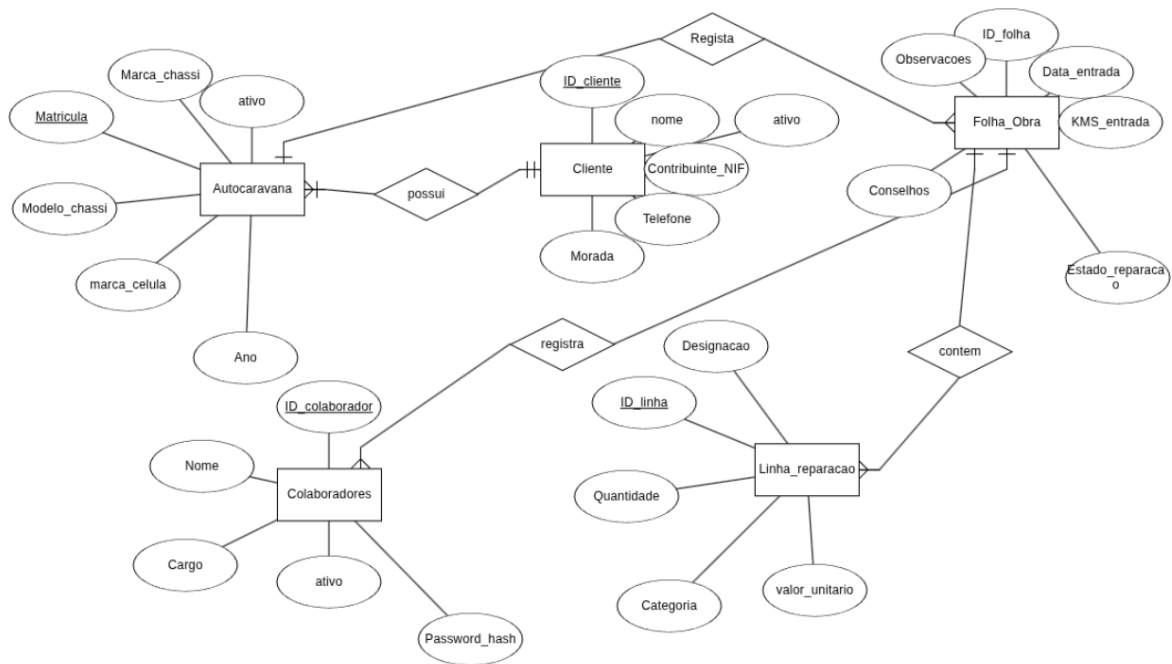


Figura 9: Diagrama concetual da primeira versão, feito no ERDPlus. A cardinalidade entre colaboradores e folhas está invertida (ver secção 3).

Anexo III: Esquema relacional (primeira versão)

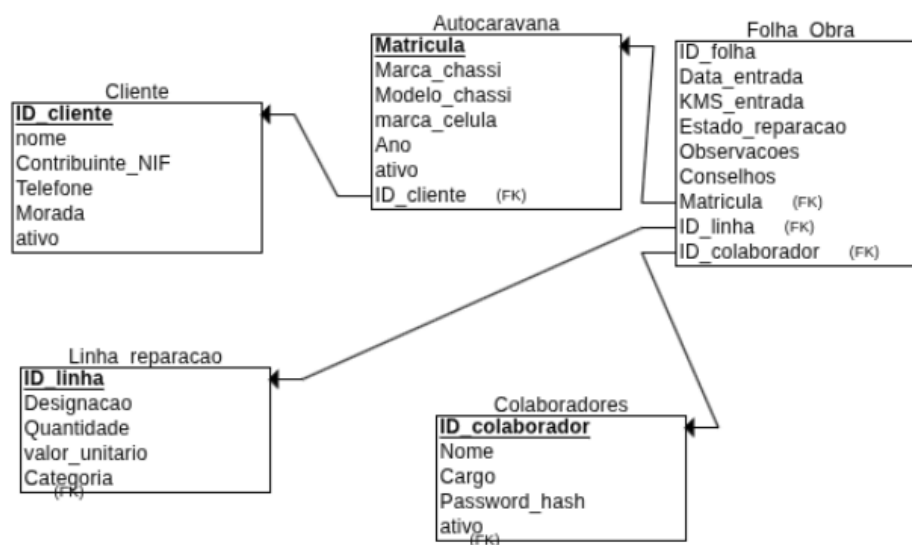


Figura 10: Esquema relacional da primeira versão, feito no ERDPlus. A chave estrangeira entre a folha e as linhas está do lado errado (ver secção 3).